

Your grade: 90%

Your latest: 90% • Your highest: 90% • To pass you need at least 80%. We keep your highest score.

[Next item →](#)

1. We use the "cache" in our implementation of forward and backward propagation to pass useful values to the next layer in the forward propagation. True/False?

1 / 1 point

- ☐ True
- ☒ False

 **Nice work**

Correct. The "cache" is used in our implementation to store values computed during forward propagation to be used in backward propagation.

2. Which of the following are “parameters” of a neural network? (Check all that apply.)

1 / 1 point

- ☐ L the number of layers of the neural network.
- ☒ $b^{[l]}$ the bias vector.

 **Nice work**

Correct. The weight matrices and the bias vectors are the parameters of the network.

- ☐ $g^{[l]}$ the activation functions.
- ☒ $W^{[l]}$ the weight matrices.

 **Nice work**

Correct. The weight matrices and the bias vectors are the parameters of the network.

Correct. The weight matrices and the bias vectors are the parameters of the network.

3. Considering the intermediate results below, which layers of a deep neural network are they likely to belong to?

1 / 1 point



- ☐ Early layers of the deep neural network.
- ☐ Middle layers of the deep neural network.
- ☒ Later layers of the deep neural network.
- ☐ Input layer of the deep neural network.

✓ Nice work

Correct. The deep layers of a neural network are typically computing more complex features such as the ones shown in the figure.

4. Vectorization allows you to compute forward propagation in an L -layer neural network without an explicit for-loop (or any other explicit iterative loop) over the layers $l=1, 2, \dots, L$. True/False?

0 / 1 point

4. Vectorization allows you to compute forward propagation in an L -layer neural network without an explicit for-loop (or any other explicit iterative loop) over the layers $l=1, 2, \dots, L$. True/False?

0 / 1 point

- ☒ True
- ☐ False

☒ Try again

Forward propagation propagates the input through the layers, although for shallow networks we may just write all the lines $(a^{[2]} = g^{[2]}(z^{[2]}), z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}, \dots)$ in a deeper network, we cannot avoid a for loop iterating over the layers: $(a^{[l]} = g^{[l]}(z^{[l]}), z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \dots)$.

5. Assume we store the values for $n^{[l]}$ in an array called `layer_dims`, as follows: `layer_dims = [nx, 4, 3, 2, 1]`. So layer 1 has four hidden units, layer 2 has 3 hidden units and so on. Which of the following for-loops will allow you to initialize the parameters for the model?

1 / 1 point

- ☒
- ```
for i in range(1, len(layer_dims)):

 parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01

 parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```
- ☐
- ```
for i in range(1, len(layer_dims)):

    parameter['W' + str(i)] = np.random.randn(layer_dims[i-1], layer_dims[i]) * 0.01

    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```
- ☐
- ```
for i in range(1, len(layer_dims)/2):

 parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01

 parameter['b' + str(i)] = np.random.randn(layer_dims[i-1], 1) * 0.01
```
- ☐
- ```
for i in range(1, len(layer_dims)/2):

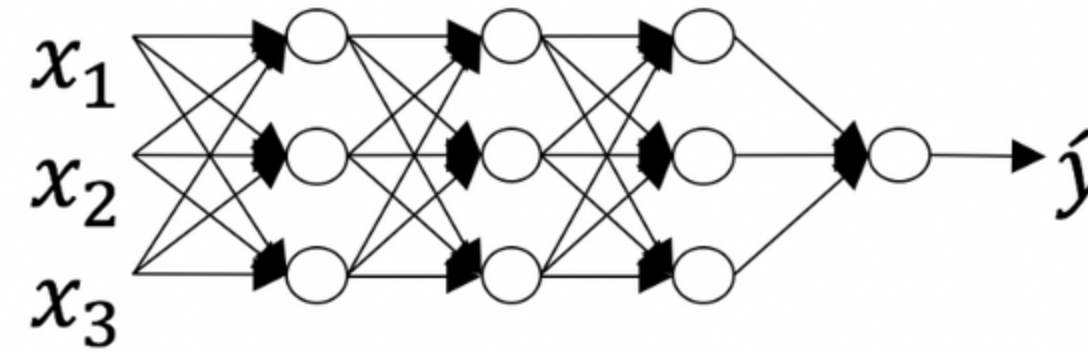
    parameter['W' + str(i)] = np.random.randn(layer_dims[i], layer_dims[i-1]) * 0.01

    parameter['b' + str(i)] = np.random.randn(layer_dims[i], 1) * 0.01
```

☑ Nice work

6. Consider the following neural network.

1 / 1 point



How many layers does this network have?

- ☐ The number of layers L is 3. The number of hidden layers is 3.
- ☐ The number of layers L is 5. The number of hidden layers is 4.
- ☒ The number of layers L is 4. The number of hidden layers is 3.
- ☐ The number of layers L is 4. The number of hidden layers is 4.

✓ Nice work

Yes. As seen in lecture, the number of layers is counted as the number of hidden layers + 1. The input and output layers are not counted as hidden layers.

7. During forward propagation, in the forward function for a layer l you need to know what is the activation function in a layer (sigmoid, tanh, ReLU, etc.). During backpropagation, the corresponding backward function also needs to know what is the activation function for layer l , since the gradient depends on it. True/False?

1 / 1 point

- ☒ True
- ☐ False

✓ Nice work

Yes, as you've seen in week 3 each activation has a different derivative. Thus, during backpropagation you need to know which activation was used in the forward propagation to be able to compute the correct derivative.

8. A shallow neural network with a single hidden layer and 6 hidden units can compute any function that a neural network with 2 hidden layers and 6 hidden units can compute. True/False?

1 / 1 point

☐ False

✓ Nice work

Yes, as you've seen in week 3 each activation has a different derivative. Thus, during backpropagation you need to know which activation was used in the forward propagation to be able to compute the correct derivative.

8. A shallow neural network with a single hidden layer and 6 hidden units can compute any function that a neural network with 2 hidden layers and 6 hidden units can compute. True/False?

1 / 1 point

☒ False

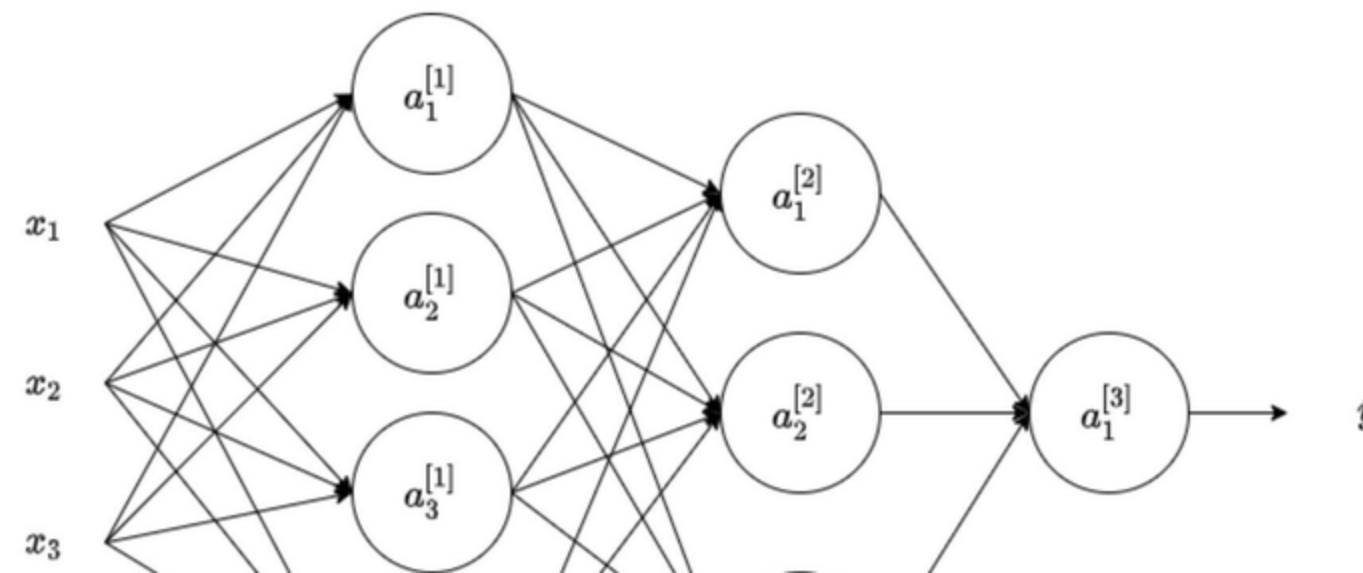
☐ True

✓ Nice work

Correct. As seen during the lectures there are functions you can compute with a "small" L-layer deep neural network that shallower networks require exponentially more hidden units to compute.

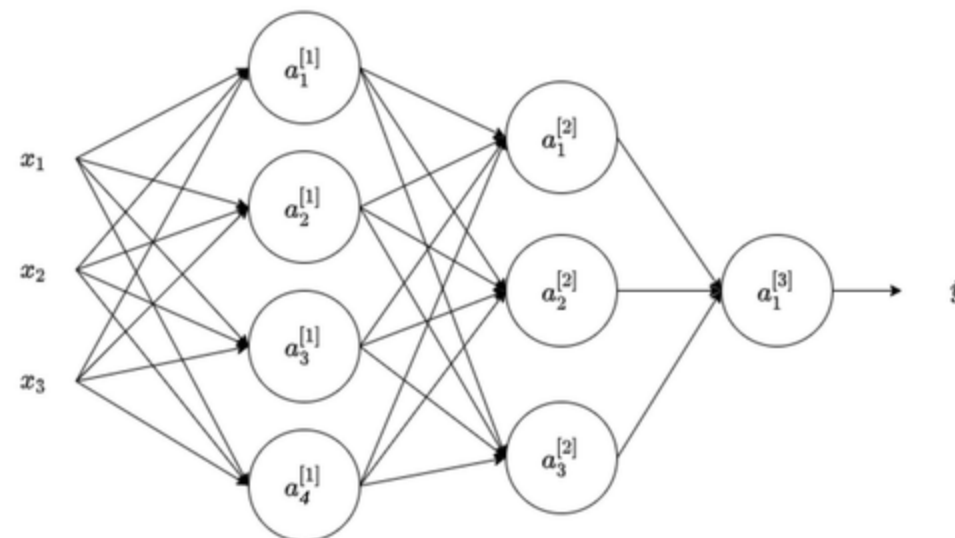
9. Consider the following 2 hidden layers neural network:

1 / 1 point



9. Consider the following 2 hidden layers neural network:

1 / 1 point



Which of the following statements is true? (Check all that apply).

- ☐ $W^{[2]}$ will have shape (1, 3)
- ☐ $W^{[2]}$ will have shape (4, 3)
- ☐ $W^{[2]}$ will have shape (3, 1)
- ☐ $W^{[1]}$ will have shape (3, 4)
- ☐ $b^{[1]}$ will have shape (3, 1)
- ☒ $W^{[1]}$ will have shape (4, 3)

Nice work

 Yes. More generally, the shape of $W^{[l]}$ is $(n^{[l]}, n^{[l-1]})$.

- ☒ $W^{[2]}$ will have shape (3, 4)

Nice work

 Yes. More generally, the shape of $W^{[l]}$ is $(n^{[l]}, n^{[l-1]})$.

- ☒ $b^{[1]}$ will have shape (4, 1)

Nice work

 Yes. More generally, the shape of $b^{[l]}$ is $(n^{[l]}, 1)$.

- ☐ $b^{[1]}$ will have shape (1, 4)

☒ $W^{[1]}$ will have shape (4, 3)

✓ Nice work

Yes. More generally, the shape of $W^{[l]}$ is $(n^{[l]}, n^{[l-1]})$.

☒ $W^{[2]}$ will have shape (3, 4)

✓ Nice work

Yes. More generally, the shape of $W^{[l]}$ is $(n^{[l]}, n^{[l-1]})$.

☒ $b^{[1]}$ will have shape (4, 1)

✓ Nice work

Yes. More generally, the shape of $b^{[l]}$ is $(n^{[l]}, 1)$.

☐ $b^{[1]}$ will have shape (1, 4)

10. In the general case if we are training with m examples what is the shape of $A^{[l]}$?

1 / 1 point

☐ $(m, n^{[l]})$

☐ $(m, n^{[l+1]})$

☐ $(n^{[l+1]}, m)$

☒ $(n^{[l]}, m)$

✓ Nice work

Yes. The number of rows in $A^{[1]}$ corresponds to the number of units in the l-th layer.